

Reporte

Paso 1

Se cargó el workload a partir del comando `python3 run_workload.py --db bookstore_g3 --csv times_baseline.csv`. A partir de eso se obtuvo un tiempo de **15025** milisegundos la primera vez. Para normalizar el caché se corrió una segunda vez obteniendo un tiempo de **14239** milisegundos.

Luego, se evaluaron las consultas y se observó que habían demasiadas consultas similares y muchas consultas compartían las mismas tablas. Vamos a considerar las consultas más importantes observadas.

1. Consultas Reviews de libros.

Es de las consultas que más apareció (~28). Hay dos tipos:

- las consultas que buscan obtener las 20 reseñas más recientes de un libro que tenga cierto `book_id` específico (~26)
- Las consultas anidadas cuentan cuántos libros cumplen que al menos tengan 10 reseñas y un rating promedio mayor a 4 (~2).

Nos centraremos principalmente en la primera que cumplen el siguiente formato:

```
SELECT review_id, user_id, rating, review_text, review_date  
FROM reviews  
WHERE book_id = 20053  
ORDER BY review_date DESC LIMIT 20;
```

El plan de estas consultas es el siguiente:

```
explain("""  
EXPLAIN ANALYZE  
SELECT review_id, user_id, rating, review_text, review_date  
FROM reviews  
WHERE book_id = 20053  
ORDER BY review_date DESC LIMIT 20;""")  
  
Limit (cost=19161.42..19163.75 rows=20 width=84) (actual time=95.213..98.845 rows=20.00 loops=1)  
  Buffers: shared hit=638 read=12388  
  → Gather Merge (cost=19161.42..19172.49 rows=95 width=84) (actual time=95.212..98.841 rows=20.00 loops=1)  
    Workers Planned: 2  
    Workers Launched: 2  
    Buffers: shared hit=638 read=12388  
    → Sort (cost=18161.40..18161.50 rows=40 width=84) (actual time=68.121..68.122 rows=17.33 loops=3)  
      Sort Key: review_date DESC  
      Sort Method: top-N heapsort Memory: 28kB  
      Buffers: shared hit=638 read=12388  
      Worker 0: Sort Method: top-N heapsort Memory: 28kB  
      Worker 1: Sort Method: top-N heapsort Memory: 28kB  
      → Parallel Seq Scan on reviews (cost=0.00..18160.33 rows=40 width=84) (actual time=0.670..67.740 rows=598.00 loops=3)  
        Filter: (book_id = 20053)  
        Rows Removed by Filter: 332735  
        Buffers: shared hit=564 read=12388  
Planning Time: 0.883 ms  
Execution Time: 98.882 ms
```

Figura 1: Plan Reviews.

PostGres lee toda la tabla entera de reviews (Parallel seq scan) buscando las filas que cumplan la condición, luego los ordena y junta los resultados porque habían workers paralelos. Tardando aproximadamente 100 ms por consulta, en total más de 2000 ms entre todas las consultas.

2. Consulta sobre correos.

Apareció ~29 veces, hay dos tipos:

- Consultas que buscan un email específico de un usuario (~21).
- Consultas que buscan un email específico de un usuario pero aplicando LOWER (~8).

Las consultas que nos centraremos son las que ocupan la función LOWER, la razón es porque el email es de tipo UNIQUE, por lo que se crea un índice automáticamente en email, pero si se aplica la función LOWER, no se está utilizando el índice para la búsqueda. Se puede observar que LOWER hace un scan completo mientras que sin LOWER ocupa un index scan.

```
> explain("""
EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, is_premium
FROM users
WHERE LOWER(email) = LOWER('ROSA.FERREIRA47314@SHOP.NET');""")

Gather (cost=1000.00..4664.71 rows=1000 width=5) (actual time=12.551..57.347 rows=1.00 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  Buffers: shared hit=1800
  → Parallel Seq Scan on users (cost=0.00..3564.71 rows=588 width=5) (actual time=14.541..34.267 rows=0.50 loops=2)
    Filter: (lower(email) = 'rosa.ferreira47314@shop.net'::text)
    Rows Removed by Filter: 100000
    Buffers: shared hit=1800
Planning Time: 0.087 ms
Execution Time: 57.382 ms
```

Figura 2: Plan LOWER

```
> explain("""
EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, is_premium, country
FROM users
WHERE email = 'Zara.moreno33740@mail.com';""")

Index Scan using users_email_key on users (cost=0.42..8.44 rows=1 width=8) (actual time=0.935..0.936 rows=1.00 loops=1)
  Index Cond: (email = 'Zara.moreno33740@mail.com'::text)
  Index Searches: 1
  Buffers: shared read=4
Planning:
  Buffers: shared hit=62 read=24
Planning Time: 9.964 ms
Execution Time: 1.051 ms
```

Figura 3: Plan email

3. Consultas JOIN order, order_items, books

Son Aproximadamente 13 y nuevamente son en dos tipos.

- Consultas que seleccionan el id del libro, el título y los ingresos, donde filtran por orden date, agrupan por books id y book title, y finalmente ordenan por ganancia (~10).
- Una consulta similar a la exterior, pero en vez de seleccionar el título seleccionan el mes y ordenan por mes (~3).

Ambas consultas son un problema, debido al triple join que hay. Las que más tiempo tardan son las del segundo tipo que tardan más de 2000 ms cada una.

```
> explain("""
EXPLAIN ANALYZE
SELECT b.book_id, b.title, b.income, o.order_date, o.order_id
FROM books b
JOIN orders o ON b.book_id = o.book_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_date >= '2023-12-01' AND o.order_date < '2023-12-01'
ORDER BY o.order_date, b.title, b.income, o.order_id;""")

Incremental Sort (cost=262437.39..437936.93 rows=1267428 width=47) (actual time=2279.224..3291.363 rows=100.00 loops=1)
  Sort Key: (data.type::text)::text, o.order_date, (data.type::text)::text, o.order_id
  Prepared Key: (data.type::text)::text, o.order_date
  Sort Method: quicksort  Memory: 2768  Peak Memory: 2768
  Buffers: shared hit=758 read=53857, temp read=36747 written=36833
  → GroupAggregate (cost=262437.39..437936.93 rows=1267428 width=47) (actual time=1895.387..3290.474 rows=100.00 loops=1)
    Group Key: (data.type::text)::text, o.order_date, b.income
    Buffers: shared hit=752 read=53857, temp read=36747 written=36833
    → Gather Merge (cost=186227.39..36427.53 rows=1267428 width=25) (actual time=185.562..2299.449 rows=192254.00 loops=1)
      Workers Planned: 2
      Buffers: shared hit=752 read=53857, temp read=36747 written=36833
      → Sort (cost=186227.39..36427.53 rows=1267428 width=25) (actual time=181.917..190.348 rows=44184.47 loops=1)
        Sort Key: (data.type::text)::text, o.order_date, b.income
        Sort Method: external merge  Memory: 87968
        Buffers: shared hit=752 read=53857, temp read=36747 written=36833
        Worker 0: Sort Method: external merge  Size: 2271205
        Worker 1: Sort Method: external merge  Size: 2271205
        → Hash Join (cost=4181.74..122739.45 rows=57188 width=25) (actual time=1801.038..1544.543 rows=44178.47 loops=1)
          Hash Cond: (b.book_id = o.order_id)
          Buffers: shared hit=72 read=53857, temp read=28145 written=28224
          → Parallel Seq Scan on order_items oi (cost=0.00..12679.45 rows=57188 width=22) (actual time=494.471..1339.718 rows=44178.47 loops=1)
            Non-Const: (oi.order_id = o.order_id)
            Buffers: shared read=204, temp read=28145 written=28224
            → Parallel Seq Scan on order_items oi (cost=0.00..16879.18 rows=2177218 width=18) (actual time=8.448..107.202 rows=177218.00 loops=1)
              Buffers: shared read=204
              → Parallel Hash (cost=2492.88..3435.88 rows=588723 width=17) (actual time=242.688..262.689 rows=39308.67 loops=1)
                Hash Cond: (b.income = oi.income)
                Buffers: shared read=19252, temp write=2188
                → Parallel Seq Scan on order_items oi (cost=0.00..16879.18 rows=2177218 width=17) (actual time=8.551..172.379 rows=39308.67 loops=1)
                  Filter: ((order_date >= '2023-12-01'::timestamp without time zone) AND (order_date < '2023-12-01'::timestamp without time zone))
                  Buffers: shared read=19252
                → Hash (cost=461.48..461.48 rows=5888 width=17) (actual time=4.204..4.204 rows=3888.00 loops=1)
                  Hash Cond: (b.income = oi.income)
                  Buffers: shared hit=70 read=0
                  → Seq Scan on books b (cost=0.00..461.48 rows=3888 width=17) (actual time=0.251..3.488 rows=3888.00 loops=1)
                    Buffers: shared hit=702 read=0
          Buffers: shared hit=752 read=53
Planning:
  Buffers: shared hit=248 read=48
Planning Time: 11.243 ms
```

Figura 4: Plan Segundo tipo de join

Se puede ver que se hacen 3 loops para el primer hash join qué tiene 464184 filas, entonces el resultado intermedio tiene más de 1.3 millones de filas, lo que tiene un gran costo, y luego de eso se reduce a 570000 aproximadamente lo que es sumamente costoso.

Ambos tipos de join tienen algo en común, filtran por `order_date`, entonces se tendrá en consideración.

4. Consultas que utilizan `order_date` para filtrar (~21).

Son varias consultas que ocupan `order_date` de la tabla `order` para poder filtrar y hacer condiciones.

Dos tipos:

- Que utilizan tres joins (las consultas de arriba) y otras consultas que son dos joins solamente.
- Consulta simple de búsqueda de encontrar órdenes entre fechas.

```
explain(***
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.order_id, o.user_id, o.order_date, o.total_amount
FROM orders o
JOIN users u ON u.user_id = o.user_id
WHERE u.is_premium = TRUE
AND o.order_date >= '2024-12-15' AND o.order_date < '2024-12-22'
ORDER BY o.order_date DESC;****)

Gather Merge (cost=38798.86..39099.93 rows=2585 width=22) (actual time=141.825..145.629 rows=2434.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=3788 read=17618
  → Sort (cost=37798.84..37881.53 rows=1877 width=22) (actual time=116.458..116.492 rows=811.33 loops=3)
    Sort Key: o.order_date DESC
    Sort Method: quicksort Memory: 4168
    Buffers: shared hit=3788 read=17618
    Worker 0: Sort Method: quicksort Memory: 5448
    Worker 1: Sort Method: quicksort Memory: 5448
    → Parallel Hash Join (cost=3124.51..37744.68 rows=1877 width=22) (actual time=6.138..115.999 rows=811.33 loops=3)
      Hash Cond: (o.user_id = u.user_id)
      Buffers: shared hit=3774 read=17618
      → Parallel Seq Scan on orders o (cost=0.00..34592.00 rows=10700 width=22) (actual time=0.889..108.195 rows=8107.33 loops=3)
        Filter: ((order_date >= '2024-12-15 00:00:00'::timestamp without time zone) AND (order_date < '2024-12-22 00:00:00'::timestamp without time zone))
        Rows Removed by Filter: 791893
        Buffers: shared hit=1974 read=17618
      → Parallel Hash (cost=2978.47..2978.47 rows=11843 width=4) (actual time=4.629..4.629 rows=6665.00 loops=3)
        Buckets: 32768 Batches: 1 Memory Usage: 185408
        Buffers: shared hit=1880
        → Parallel Seq Scan on users u (cost=0.00..2976.47 rows=11843 width=4) (actual time=0.807..11.481 rows=19995.00 loops=1)
          Filter: is_premium
          Rows Removed by Filter: 180805
          Buffers: shared hit=1880
Planning:
  Buffers: shared hit=8
Planning Time: 8.192 ms
Execution Time: 145.773 ms
```

Figura 5: Plan de join simple con order

Se puede observar que se hace seq scan para poder ver los filtros de order.

A partir de toda la información obtenida, está claro las consultas que hay que optimizar. En primer lugar hay que optimizar las consultas que ocupan muchos joins, ya que poseen muchas cantidad de tuplas del resultado de intermedio y son las que más tiempo se demoran. Luego todas las consultas entregas de manera similar, ya que a pesar que no demoran tanto tiempo como los joins, son tantas, que en su cúmulo, ocupan tiempo.

Hay otras consultas presentes que no se tomarán tanto en consideración, por ejemplo el de stock de inventario, debido a que se hacen buscando en sus llaves primarias y el tiempo que utilizan no son relevantes. Otras consultas son en libros, no son tantas y son muy específicas, entonces no llevaría afectar más de algún tipo de consulta

PASO 2 MATERIALIZE:

Antes de ver qué índices proponer vamos a ver que debemos materializar. Como se pudo observar, son sumamente costosas las consultas que involucran un triple join ya que el resultado intermedio de los joins es sumamente caro al calcularlo entonces conveniente. La única diferencia entre las consultas reescritas y las originales, es que en vez de hacer el triple join, se utiliza la tabla creada directamente por el `MATERIALIZED`

```

CREATE MATERIALIZED VIEW join_orders_items_books AS
SELECT o.order_date,
       b.book_id, b.title, b.genre,
       oi.quantity * oi.unit_price AS line_revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN books b      ON b.book_id = oi.book_id;

```

¿Por qué los índices no son suficientes?

Por lo que se observó en el EXPLAIN de los pasos el primer HASH JOIN del segundo tipo de consulta de los triple joins, genera más de 1.300.000 filas. Un índice no elimina el costo principal que es el del producto intermedio.

Costo de mantención

Como la vista materializada deja la tabla como esta, si los datos de las tablas utilizadas en los joins cambian, sería necesario ejecutar REFRESH. Pero en este caso, los datos son estáticos por lo que el costo del refresco es cero.

PASO 3: ÍNDICES

Una vez clasificadas las consultas se proponen índices y creado la tabla de materialized.

1)

CREATE INDEX ON reviews (book_id, review_date DESC);

Cubre ~26 consultas de reviews que filtran por book_id exacto y ordenan por review_date DESC. El índice compuesto evita tanto el scan completo de la tabla como el sort posterior, ya que el orden del índice coincide exactamente con el ORDER BY de las queries.

2)

CREATE INDEX ON orders (order_date);

Cubre ~20 consultas que filtran orders por rangos de fecha (order_date BETWEEN). Aplica tanto a queries directas sobre orders como a los triple JOIN con order_items y books, que son las consultas más costosas del workload por involucrar múltiples tablas.

3)

CREATE INDEX ON users (LOWER(email));

Cubre las ~7 consultas que usan LOWER(email) = LOWER('...'), las cuales no pueden usar el índice B-Tree creado automáticamente por el UNIQUE constraint de email, ya que ese índice no indexa el resultado de la función LOWER.

4)

CREATE INDEX ON materialized (order_date);

Este índice optimiza las consultas más pesadas del workload que involucran el triple JOIN, permitiendo filtrar eficientemente por rangos de fecha sobre el producto cruz de las tablas.

Paso 4

Utilizando explain comparamos las consultas sin índices y con índices

Una vez aplicados los índices y la vista materializada se mide el tiempo de ejecución y se alcanzan 3597 milisegundos.

```
makia@andr  ~$ python3 test_copia_upgrade.py -3.12 run_workload.py --db bookstore_g3_copy --workload workload_after.sql
1 --user postgres --password postgres --csv times_baseline.csv
Connected to bookstore_g3_copy. Running 100 queries from workload_after.sql.

>>> T_workload = 3597 ms <<<

Wrote per-query timings to times_baseline.csv
```

Figura 6: Tiempo final

A continuación se presenta alguno de los planes que utilizaron las consultas optimizadas (Se pueden ver mejor en los archivos extras entregados):

```
explain("""
EXPLAIN ANALYZE
SELECT review_id, user_id, rating, review_text, review_date
FROM reviews
WHERE book_id = 20653
ORDER BY review_date DESC LIMIT 20;""")

Limit (cost=0.42..81.43 rows=20 width=84) (actual time=0.029..0.048 rows=20 loops=1)
  Buffers: shared hit=23
  → Index Scan using reviews_book_id_review_date_idx on reviews (cost=0.42..378.03 rows=93 width=84) (actual time=0.028..0.045 rows=20 loops=1)
    Index Cond: (book_id = 20653)
    Index Searches: 1
    Buffers: shared hit=23
Planning Time: 0.155 ms
Execution Time: 0.048 ms
```

Figura 7: Plan Review AFTER

```
explain("""
EXPLAIN ANALYZE
SELECT genre, DATE_TRUNC('month', order_date) AS month,
SUM(line_revenue) AS revenue
FROM join_orders_line_books
WHERE order_date >= '2022-12-01' AND order_date < '2023-12-01'
GROUP BY genre, DATE_TRUNC('month', order_date)
ORDER BY month, revenue DESC;
""")

Sort (cost=292178.62..292173.18 rows=438031 width=47) (actual time=754.835..754.841 rows=188.00 loops=1)
  Sort Key: (date_trunc('month', order_date)), (sum(line_revenue)) DESC
  Sort Method: quicksort Memory: 320
  Buffers: shared read=50796
  → HashAggregate (cost=186576.81..211023.41 rows=438031 width=47) (actual time=754.673..754.750 rows=188.00 loops=1)
    Group Key: date_trunc('month', order_date), genre
    Planned Partitions: 12 Batches: 1 Memory Usage: 609KB
    Buffers: shared read=50796
    → Bitmap Heap Scan on join_orders_line_books (cost=21076.81..92114.85 rows=1270105 width=71) (actual time=75.493..446.488 rows=1271313.00 loops=1)
      Recheck Cond: (order_date >= '2022-12-01 00:00:00'::timestamp without time zone) AND (order_date < '2023-12-01 00:00:00'::timestamp without time zone)
      Heap Blocks: exact=4726
      Buffers: shared read=50796
      → Bitmap Index Scan on join_orders_line_books_order_date_idx (cost=0.00..20757.48 rows=1270105 width=0) (actual time=0.361..49.341 rows=1271313.00 loops=1)
        Index Cond: ((order_date >= '2022-12-01 00:00:00'::timestamp without time zone) AND (order_date < '2023-12-01 00:00:00'::timestamp without time zone))
        Index Searches: 1
        Buffers: shared read=2040
Planning Time: 0.151 ms
Execution Time: 755.302 ms
```

Figura 8: Triple Join AFTER

```
explain("""
EXPLAIN ANALYZE
SELECT user_id, is_premium
FROM users
WHERE LOWER(email) = LOWER('ROSA.FERREIRA47314@SHOP.NET');""")

(12)
---
Bitmap Heap Scan on users (cost=35.95..1615.44 rows=972 width=5) (actual time=0.307..0.308 rows=1.00 loops=1)
  Recheck Cond: (lower(email) = 'rosa.ferreira47314@shop.net'::text)
  Heap Blocks: exact=1
  Buffers: shared read=4
  → Bitmap Index Scan on users_lower_idx (cost=0.00..35.71 rows=972 width=0) (actual time=0.179..0.179 rows=1.00 loops=1)
    Index Cond: (lower(email) = 'rosa.ferreira47314@shop.net'::text)
    Index Searches: 1
    Buffers: shared read=3
Planning Time: 0.132 ms
Execution Time: 0.344 ms
```

Figura 8: LOWER AFTER



Figura 9: ORDER DATE AFTER

Tabla 1: Tiempos de ejecución por clase de consulta representativa (ms)

Clase de consulta	Indice aplicado	Query	Baseline (ms)	Optimizado (ms)	Reduccion
Consultas Reviews — WHERE book_id = X ORDER BY review_date DESC LIMIT 20					
Q-reviews	reviews(book_id, review_date DESC)	036	147.15	0.50	99.7%
Q-reviews	reviews(book_id, review_date DESC)	083	146.33	1.32	99.1%
Consultas email con LOWER — WHERE LOWER(email) = LOWER('...')					
Q-users-lower	users(LOWER(email))	032	55.36	0.57	99.0%
Q-users-lower	users(LOWER(email))	065	64.86	0.19	99.7%
Triple JOIN — GROUP BY genre + DATE_TRUNC('month') (rango anual)					
Q-triple-genre	join_orders_items_books(order_date)	033	2144.68	472.87	78.0%
Q-triple-genre	join_orders_items_books(order_date)	057	2352.84	523.39	77.8%
Triple JOIN — GROUP BY book_id, title (rango mensual)					
Q-triple-revenue	join_orders_items_books(order_date)	042	398.68	130.15	67.4%
Q-triple-revenue	join_orders_items_books(order_date)	059	402.91	141.06	65.0%
Consultas order_date — busqueda directa por rango de fecha					
Q-orders-range	orders(order_date)	027	92.32	1.88	98.0%
Q-orders-range	orders(order_date)	084	144.55	1.87	98.7%
Consultas order_date — JOIN con users WHERE is_premium = TRUE					
Q-orders-premium	orders(order_date)	056	139.17	61.90	55.5%
Q-orders-premium	orders(order_date)	094	143.30	63.62	55.6%
TOTAL WORKLOAD (100 consultas)			14.237	3.594	74.8%

(*) Mediciones en caliente (segunda ejecucion). Tiempos en milisegundos. Speedup total: 3.96x.